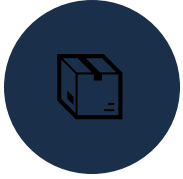


3강

Python 기초 문법

변수 · 자료형 · 리스트 · 딕셔너리 · 조건문 · 반복문 · 함수

오늘의 목표



변수

데이터를 저장하는
이름 있는 상자



자료형

숫자, 문자열,
참/거짓 구분하기



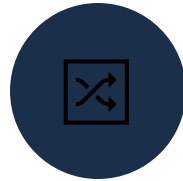
리스트

여러 데이터를
순서대로 담기



딕셔너리

이름표가 붙은
데이터 저장소



조건문

"만약 ~이면"
분기 처리하기



반복문

같은 작업을
여러 번 자동 반복

변수 (Variable) — 데이터를 담는 상자

변수란?

데이터를 저장해두는 이름 있는 공간입니다.
변수에 값을 넣으면 나중에 꺼내 쓸 수 있어요.

name	"김민수"
------	-------

age	20
-----	----

score	92
-------	----

코드로 보면:

```
# 변수에 값 저장하기
name = "김민수"      # 문자열
age = 20              # 숫자
score = 92            # 숫자

# 저장된 값 출력하기
print(name)           # 김민수
print(age)            # 20
print(score)          # 92

# 변수 활용
print(name + "의 점수:", score)
```

변수 — 이름 짓는 규칙 & 값 바꾸기

변수 이름 규칙

`score = 90`

영문·숫자·언더바(_) 사용 가능 ✓

`my_score = 90`

단어 사이에 언더바 권장 ✓

`1score = 90`

숫자로 시작 불가 ✕

`if = 90`

예약어(if, for, def ...) 불가 ✕

`my-score = 90`

하이픈(-) 사용 불가 ✕

변수 값은 언제든지 다시 할당할 수 있어요!

```
# 처음 값 저장
score = 50
print(score)           # 50

# 값 바꾸기 (재할당)
score = 90
print(score)           # 90

# 계산 결과로 바꾸기
score = score + 10
print(score)           # 100

# 여러 변수 한 줄에 초기화
a, b, c = 1, 2, 3
print(a, b, c)         # 1 2 3
```

자료형 (Data Type) — 데이터의 종류

`int`

정수

```
100, -5, 0
```

소수점 없는 숫자

`type()` 확인

```
type(100)  
→ <class 'int'>
```

`float`

실수

```
3.14, 92.5
```

소수점 있는 숫자

`type()` 확인

```
type(3.14)  
→ <class 'float'>
```

`str`

문자열

```
"김민수", "A+"
```

따옴표로 감싼 텍스트

`type()` 확인

```
type("안녕")  
→ <class 'str'>
```

`bool`

참/거짓

```
True, False
```

조건의 참/거짓 값

`type()` 확인

```
type(True)  
→ <class 'bool'>
```

자료형 변환 (Type Conversion) — 형태를 바꾸는 함수

자료형은 필요에 따라 서로 변환할 수 있어요.

특히 `input()`으로 받은 값은 항상 문자열(`str`)이라 계산하려면 변환이 필요합니다!

`int()`

실수·문자열 → 정수

```
int("90") → 90  
int(3.7) → 3
```

`float()`

정수·문자열 → 실수

```
float("3.14") → 3.14  
float(9) → 9.0
```

`str()`

숫자 → 문자열

```
str(90) → "90"  
str(3.14) → "3.14"
```

`bool()`

숫자·문자열 → 참/거짓

```
bool(0) → False  
bool(1) → True
```

💡 `input()` 활용: `age = int(input("나이: "))` → 이제 숫자로 계산 가능! (`input` 없이는 항상 `str`)

리스트 (List) — 여러 데이터를 순서대로 담기

리스트는 여러 개의 값을 하나의 변수에 담을 수 있어요.
각 값에는 0부터 시작하는 번호(인덱스)가 붙습니다.

[0]	[1]	[2]	[3]	[4]
85	90	78	92	88

```
scores = [85, 90, 78, 92, 88]
```

```
scores = [85, 90, 78, 92, 88]

print(scores)          # [85, 90, 78, 92, 88]
print(scores[0])       # 85   (첫 번째)
print(scores[-1])      # 88   (마지막)
print(scores[1:3])     # [90, 78] (슬라이싱)
print(len(scores))     # 5   (개수)

scores.append(95)       # 끝에 추가
scores.remove(78)       # 값으로 삭제
```

리스트 메서드 — 데이터 추가 · 삭제 · 정렬

자주 쓰는 메서드

<code>append(값)</code>	끝에 항목 추가
<code>insert(i, 값)</code>	i번 위치에 삽입
<code>remove(값)</code>	값으로 항목 삭제 (첫 번째)
<code>pop()</code>	마지막 항목 꺼내기
<code>sort()</code>	오름차순 정렬 (원본 변경)
<code>len(리스트)</code>	항목 개수 반환

직접 써보기

```
scores = [85, 90, 78]

scores.append(92)
# → [85, 90, 78, 92]

scores.insert(1, 88)
# → [85, 88, 90, 78, 92]

scores.remove(78)
# → [85, 88, 90, 92]

scores.sort()
# → [85, 88, 90, 92]

print(len(scores))    # 4
print(max(scores))    # 92
print(sum(scores))    # 355
```


딕셔너리 (Dictionary) — 이름표가 붙은 데이터

딕셔너리는 key(이름)와 value(값)를 쌍으로 저장해요.
리스트가 번호로 찾는다면, 딕셔너리는 이름으로 찾아요.

이름	"김민수"
나이	20
점수	92
학과	"컴퓨터공학"

```
student = {  
    "이름": "김민수",  
    "나이": 20,  
    "점수": 92,  
    "학과": "컴퓨터공학"  
}  
  
# 값 가져오기  
print(student["이름"])    # 김민수  
print(student["점수"])    # 92  
  
# 값 변경 / 추가  
student["점수"] = 95  
student["학년"] = 2  
  
# 전체 키 목록  
print(student.keys())
```

딕셔너리 메서드 — 키와 값을 다루는 방법

주요 메서드

<code>.keys()</code>	모든 키 목록 반환
<code>.values()</code>	모든 값 목록 반환
<code>.items()</code>	(키, 값) 쌍 목록 반환
<code>.get(키, 기본값)</code>	키 없어도 에러 안 남
<code>키 in 딕셔너리</code>	키 존재 여부 확인 (True/False)

실제로 써보기

```
student = {
    "이름": "김민수",
    "나이": 20,
    "학과": "컴퓨터공학"
}
print(student.keys())
# dict_keys(['이름', '나이', '학과'])
print(student.values())
# dict_values(['김민수', 20, '컴퓨터공학'])
# (키, 값) 쌍으로 반복
for key, val in student.items():
    print(key, ":", val)
# 안전하게 값 가져오기
age = student.get("나이", 0)
# 키 존재 확인
print("이름" in student)    # True
print("점수" in student)    # False
```

조건문 (if/elif/else) — 상황에 따라 다르게 실행

"만약 점수가 90 이상이면 A, 80 이상이면 B, 그렇지 않으면 C를 출력해!"

```
score = 85
```

```
score >= 90 ?
```

X False

✓ True

```
score >= 80 ?
```

```
print("A")
```

B

C

```
score = 85

if score >= 90:
    print("A")          # 90점 이상
elif score >= 80:
    print("B")          # 80점 이상
elif score >= 70:
    print("C")          # 70점 이상
else:
    print("D")          # 70점 미만

# 결과: B

# 실행되는 블록만 들여쓰기 (탭)!
# → 들여쓰기가 Python의 핵심!
```

복합 조건 — and / or / not 연산자

and

두 조건 모두 True일 때만 True

```
score >= 80 and score < 90
```

or

둘 중 하나라도 True면 True

```
score < 0 or score > 100
```

not

True↔False 를 뒤집음

```
not is_passed
```

```
score = 85
# and: 두 조건 모두 만족
if score >= 80 and score < 90:
    print("B 학점")    # 출력됨
# or: 하나라도 만족
if score < 60 or score > 95:
    print("특별 처리") # 출력 안됨
# not: 조건 반전
is_passed = score >= 60
if not is_passed:
    print("재수강")    # 출력 안됨
print(not is_passed)  # False
```

반복문 (for) — 같은 작업을 여러 번 자동으로

"같은 코드를 매번 새로 쓰지 않아도 됩니다!"

range() 사용하기

```
# 0부터 4까지 (총 5번)
for i in range(5):
    print(i)
# 출력: 0 1 2 3 4

# 1부터 5까지
for i in range(1, 6):
    print(i)
# 출력: 1 2 3 4 5
```

리스트 반복하기

```
scores = [85, 90, 78, 92, 88]

# 점수 하나씩 출력
for score in scores:
    print(score)

# 조건과 함께 사용
for score in scores:
    if score >= 90:
        print(score, "→ A등급")
```

💡 for 반복문 = 리스트의 모든 데이터를 자동으로 처리할 때 사용 → 데이터 분석의 핵심!

while 반복문 & break / continue

while — 조건이 참인 동안 반복

```
# for: 횟수가 정해진 반복
for i in range(5):
    print(i)    # 0 1 2 3 4

# while: 조건이 참인 동안 반복
count = 1
while count <= 5:
    print(count)
    count = count + 1
# → 1 2 3 4 5

# 주의! 조건이 항상 참이면
# 무한루프가 됩니다 ⚠
```

for vs while

for → 반복 횟수를 미리 알 때
while → 조건이 언제 끝날지 모를 때 (사용자 입력, 게임 루프 등)

break & continue — 반복 제어

```
scores = [80, 92, 55, 88, 70]

# break: 조건 만족 시 반복 즉시 종료
for score in scores:
    if score < 60:
        print("과락 발견! 중단")
        break
    print(score)
# → 80 92 과락 발견! 중단

# continue: 해당 항목 건너뛰고 계속
for score in scores:
    if score < 60:
        continue    # 55는 건너뛴
    print(score)
# → 80 92 88 70
```

함수 (Function) — 자주 쓰는 코드를 묶어서 재사용

함수 구조

```
def 함수이름 (매개변수) :
```

```
# 실행할 코드 (들여쓰기 필수!)  
결과 = 계산  
return 결과
```

def

함수 정의 시작

매개변수

입력받는 값

return

결과값 반환

실제 코드 예시

```
# 함수 정의  
def get_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    else:  
        return "C"
```

```
# 함수 호출 (여러 번 재사용!)  
print(get_grade(95))    # A  
print(get_grade(82))    # B  
print(get_grade(74))    # C
```

```
# 함수 없이는?  
# → 같은 if문을 매번 다시 써야 함 😞
```

함수 활용 — 내장 함수 & 기본값 매개변수

Python 내장 함수 (Built-in Functions)

`print()`

화면 출력

`print("안녕")`

`len()`

길이 반환

`len([1,2,3]) → 3`

`max()`

최댓값

`max([80,90]) → 90`

`min()`

최솟값

`min([80,90]) → 80`

`sum()`

합계

`sum([80,90]) → 170`

`int()`

정수 변환

`int("90") → 90`

기본값 매개변수 — 인자를 안 넣어도 동작!

```
# 기본값을 설정하면 인자 없이도 호출 가능
def greet(name, msg="안녕하세요!") :
    print(name + ": " + msg)
```

```
greet("김민수")           # 김민수: 안녕하세요!
greet("이서연", "반갑습니다!") # 이서연: 반갑습니다!
```


이론 정리 — 오늘 배운 핵심 개념

변수	<pre>name = "김민수"</pre>	데이터를 저장하는 공간
자료형	<pre>int / float / str / bool</pre>	데이터의 종류를 구분
리스트	<pre>scores = [85, 90, 78]</pre>	순서 있는 데이터 묶음, [인덱스]로 접근
딕셔너리	<pre>{"키": 값}</pre>	이름표(key)로 데이터를 저장·조회
조건문	<pre>if / elif / else:</pre>	조건에 따라 다른 코드 실행
반복문	<pre>for i in range():</pre>	같은 코드를 여러 번 자동 반복
함수	<pre>def 이름 (매개변수):</pre>	코드를 묶어 이름 붙여 재사용